A MINI PROJECT REPORT ON

CUBESOLVE AI

A Mini Project submitted to Jawaharlal Nehuru Technological University, Kakinda

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE & DATA SCIENCE



Submitted By

S . P RASANNA KAVYA (24KT1A5449)

S . P URNIMA (24KT1A5451)

T . SWETHAPRIYA (24KT1A5453)

T. GEETHA VAISHNAVI (24KT1A5455)

Under the guidance of

Ms. K. SHELMA

Assistant Professor

Dept of ARTIFICIAL INTELLIGENCE & DATA SCIENCE

POTTI SRIRAMULU CHALAVADI MALLIKARJUNA RAO COLLEGE OF ENGINEERING & TECHNOLOGY

(Approved by AICTE, Affiliated to JNTU Kakinda)
KOTHAPETA, VIJAYAWADA-520001,A.P.

KOTHAPETA, VIJAYAWADA -520001, A.P.

2025-2026

POTTI SRIRAMULU CHALAVADI MALLIKARJUNA RAO COLLEGE OF ENGINEERING & TECHNOLOGY

Kothapeta, Vijayawada, Krishna Dt. A.P, India.

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

CERTIFICATE

This is to certify that the mini project work entitled **"CUBESOLVE AI"** is a bonafide work carried out by **"Ms. S. PRASANNA KAVYA (24KT1A5449), Ms. S. PURNIMA (24KT1A5451),Ms.T.SWETHAPRIYA(24KT1A5453), Ms.T.GEETHA VAISHNAVI (24KT1A5455)" Bachelor of Technology in Artificial Intelligence & Data Science** during the year 2025-2026.

Project Guide **Head of the Department**

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all the people who supported and guided us throughout the development of this mini project.

First and foremost, we thank our project guide Ms. K. Shelma, Assistant Professor, Department of Artificial Intelligence and Data Science, for her constant support, valuable suggestions, and encouragement. Her guidance helped us to understand the problem clearly and build a proper solution.

We would also like to thank the Head of the Department and all the faculty members of the Department of Artificial Intelligence and Data Science for providing us with the necessary resources and a good learning environment. We are grateful to the college management of Potti Sriramulu Chalavadi Mallikarjuna Rao College of Engineering & Technology for giving us the opportunity to work on this project.

Finally, we thank our friends who motivated and supported us throughout this project work. This project would not have been possible without everyone's help and cooperation.

S. Prasanna Kavya (24KT1A5449)
S. Purnima (24KT1A5451)
T. Swethapriya (24KT1A5453)
T. Geetha Vaishnavi (24KT1A5455)

Abstract

In this project, we present a client-side web application named CubeSolve AI that scans a 3×3 Rubik's Cube with a device camera, reads the colours of all six faces, checks that the resulting cube is physically valid, computes a solution, and guides the user through every move with an animated 3D cube. The application is built with React, Vite and Tailwind CSS and uses the open-source cubing library for the cube model, the solving algorithm and the 3D viewer. The whole application runs inside the browser. It has no back-end server, no database, no user login, and no image is ever uploaded. Colour recognition is done without OpenCV or any machine-learning model. Each of the nine stickers of a face is sampled from its cell, converted from sRGB to the CIE Lab colour space and matched to the nearest of the six centre stickers, which act as colour references for that scan. In this way the result adapts to the lighting and to cubes that do not use the standard colour scheme. Stickers that are far from every reference, or almost equally close to two references, are flagged as uncertain so that the user can correct them manually on a review screen. Before solving, the scanned state is validated against the rules of a real cube: each colour must appear exactly nine times, centres must be correct, every corner and edge must be a valid and unique piece, corner twists, edge flips and permutation parity must be possible. Plain-language error messages explain any problem. Only a valid state is sent to the solver, which runs in a Web Worker, and every generated solution is verified on the cube model before it is shown. The result is a simple, private and free tool that can be used from any modern browser, and it shows how computer vision, colour science and puzzle-solving algorithms can be combined entirely on the client side.

1. INTRODUCTION

● ABOUT THE PROJECT

Have you ever picked up a scrambled Rubik's Cube and had no idea where to begin? CubeSolve AI is built for exactly that moment. Hold the cube in front of your camera, show each of its six faces, and the application reads the colours, checks them, and tells you the moves needed to solve the cube, one at a time, with a 3D cube that turns along with you. If you do not want to use a camera, you can upload a photo of each face instead. Price: Free.

Application Features

- Camera scanning of all six faces with an on-screen guide square, or photo upload for each face.
- Automatic colour recognition using centre-sticker calibration in CIE Lab colour space.
- Review screen with a cube net, uncertain stickers marked in amber, tap-to-correct colours and 90° face rotation.
- Plain-language validation messages for impossible cube states.
- Verified solution with step-by-step controls: Next, Previous, Reset and Auto-play.

Currently Supported Platforms

All modern web browsers (Chrome, Firefox, Edge, Safari) on desktop and tablet devices.

Genre

Computer Vision, Puzzle Solving, Educational Tool, Web Application.

● PROBLEM STATEMENT

Design a web application that can be opened in a browser without installation or registration, and that lets a user solve a scrambled 3×3 Rubik's Cube by simply showing the cube to a camera. The application must read the colours reliably under normal lighting, detect and explain scanning mistakes, refuse impossible cube states, and present a correct solution in a form that a beginner can follow. It must not require server infrastructure and must not send the user's images anywhere.

- **OBJECTIVES OF STUDY**

  - To capture the six faces of a Rubik's Cube using a browser camera or uploaded photos.
  - To classify sticker colours accurately using Lab colour distance with the centre stickers as references, without heavy computer-vision libraries.
  - To validate the scanned cube state mathematically (counts, pieces, twists, flips and parity) and report problems in plain language.
  - To compute and verify a solution using a Kociemba-style two-phase solver running in a Web Worker.
  - To guide the user step by step through the solution with an animated 3D view.

- **PROJECT DEFINITION**

CubeSolve AI is a single-page client-side web application built with React 19, Vite and Tailwind CSS, with a hash-based router and no server dependency. The user interacts through mouse, keyboard or touch to scan faces, review colours, generate a solution and step through it. The scan state (captures, rotations and manual corrections) is kept in the browser's sessionStorage so a page refresh does not lose progress, and it is discarded when the tab is closed.

2. RELATED WORK

- **EXISTING SYSTEM**

Manual colour entry solvers

Many online Rubik's Cube solvers ask the user to click on each of the 54 stickers and choose its colour. This is slow, tiring and error-prone, and a single wrong click gives an invalid cube with little explanation.

Mobile apps with scanning

Some mobile apps scan the cube with the camera, but they usually need installation, may upload images to a server, and often fail silently when the lighting is poor.

Heavy computer-vision approaches

Research projects often rely on OpenCV or neural networks. These give high accuracy but add large dependencies, training data and hardware requirements that are unnecessary for a simple educational tool.

Solver libraries without a user interface

Algorithm libraries can solve a cube from a given state, but a beginner cannot use them without writing code.

- **REQUIREMENT ANALYSIS**

The primary requirement of this application is a modern web browser with camera support. CubeSolve AI is a free, client-side tool. To develop or modify it, Node.js and npm are needed. The project uses plain JavaScript and React, so learners can follow the structure easily and extend it, for example with a better colour classifier or a different solver.

- **SOFTWARE REQUIREMENTS**

  - Visual Studio Code
  - Node.js 22.12 or newer, with npm
  - Google Chrome
  - React 19.3.0, Vite 8.3.2, Tailwind CSS, cubing 0.63.8

- **SYSTEM REQUIREMENT SPECIFICATION**

The system requirement for running the application is any device with a modern browser. All application logic is executed by the browser's JavaScript engine. To use the live camera, the page must be opened from a secure context: http://localhost, http://127.0.0.1 or an HTTPS address. If it is opened from another device over a plain-HTTP local network address, the browser blocks the camera; in that case the Upload photo option can be used. An internet connection is not required once the page is loaded. On a device where JavaScript is disabled, the application will not work. The requirements for running CubeSolve AI are:

- Latest stable Chrome, Edge or Firefox
- Any webcam or phone camera
- localhost, 127.0.0.1 or HTTPS
- Node.js - 22.12 or newer

- **PROPOSED SYSTEM**

CubeSolve AI delivers its whole workflow inside the browser. As a purely client-side system it has natural limitations, such as approximate colour recognition and no storage of scans across sessions, and these are listed in the Limitations section as the roadmap for future work.

Capabilities

- Access to the device camera through the browser MediaDevices API, with a clear message if permission is blocked.
- Photo upload for each face as an alternative to the camera.
- Median sampling of the central 50% of each sticker cell, so that glare and shadow edges have little effect.
- Colour classification in CIE Lab space with centre stickers as references and flagging of uncertain stickers.
- Manual correction of any sticker and 90° rotation of a whole face on the review screen.
- Full mathematical validation of the cube state with plain-language error messages.
- Solving with cubing library's 3×3×3 solver inside Web Worker so interface never freeze.
- Verification of the solution on the cube model before display.
- Step-by-step guidance with Next, Previous, Reset and Auto-play, an animated 3D cube (TwistyPlayer) and a highlighted layer on the net.
- Scan state persistence in sessionStorage during the browser tab's lifetime.

3. DESIGNER

React lets the developer describe the interface as a tree of components. Each component receives properties and keeps its own state, and the page is updated automatically when that state changes. In CubeSolve AI the components are written in JSX and styled with Tailwind CSS utility classes. The project has two types of building blocks: visible UI components such as buttons, the face chips, the cube net and the 3D viewer, and non-visible logic modules such as the cube model, the validator, the colour classifier and the solver, which are plain JavaScript functions that can be tested without a browser.

The application is divided into four pages, each of which is a React component: Home, Scan, Review and Solve. They share a common Layout component with the header navigation and a footer that states that processing runs in the browser and no images are uploaded.

• OUR PROJECT'S DESIGN

First, we planned the three user stages (scan, review, solve) and decided that the application should be a single page with a header containing the links Home, Scan, Review and Solve. The Home page introduces the app and has a Start scanning button. The Scan page has a row of six face chips that show which faces are captured, a Camera / Upload photo tab, a live preview with a square guide divided into a 3×3 grid, and a side panel showing the captured face and the overall progress. The Review button remains disabled until all six faces are captured.

The Review page shows the six faces as a cube net. Each sticker is a button; uncertain stickers have an amber outline. Tapping a sticker opens a colour chooser. Each face has Rotate and Retake controls. A validation panel at the side either says that the cube state is valid or lists the problems found, and the Generate solution button is enabled only when the state is valid.

The Solve page has the 3D cube with a progress bar and the control buttons on the left, and three cards on the right: the current instruction, the cube state net, and the list of all moves. A "Verified solved" badge appears when the final state is solved.

Face and Colour Convention

- A cube state is an array of 54 face letters in the order U R F D L B, nine per face, read row by row as seen from outside the cube. This is the standard Kociemba facelet order.
- The centre sticker of each face is the colour reference of that face. A scanned sticker takes the letter of the nearest centre in Lab space.
- The default labels follow the Western colour scheme: Up white, Right red, Front green, Down yellow, Left orange, Back blue.

• COMPONENTS STRUCTURE

Component	Description
Layout	Header with logo and navigation links (Home, Scan, Review, Solve) and a footer with the privacy note
Home page	Introduction to the app and the Start scanning button
Scan page	Face chips, Camera / Upload photo tabs, live preview, capture button and progress panel
CameraCapture	Starts the camera stream, draws the guide square and captures the current frame

Component	Description
Review page	Cube net with correctable stickers, rotate and retake controls, warnings
CrossNet	Draws the six faces in the cross-shaped net and highlights the layer being turned
ValidationPanel	Shows “Cube state is valid” or the list of problems found
Solve page	Generate solution, step controls, progress bar, move list and state net
CubeViewer	Wraps the cubing TwistyPlayer to show the animated 3D cube

4. BLOCKS (CODE LOGIC)

Every component in CubeSolve AI has properties whose current values form its state. For example, a sticker’s state is the face letter it represents and whether it is uncertain; the Solve page’s state is the list of moves and the index of the current move. The initial scan state is loaded from sessionStorage when the page opens. Below are the main logic blocks of the application.

- **OUR PROJECT BLOCK**

1. Sampling a face

The sampleFace() function reads the nine stickers inside the guide square. For each cell it takes the central 50% of the cell and returns the per-channel median of the pixels, so reflections and dark edges between stickers do not change the result.

```
export function sampleFace(imageData, rect) {
  const { data, width } = imageData;
  const cell = rect.size / 3;
  const out = [];
  for (let row = 0; row < 3; row++) {
    for (let col = 0; col < 3; col++) {
      const x0 = Math.round(rect.x + col * cell + cell * 0.25);
      const y0 = Math.round(rect.y + row * cell + cell * 0.25);
      const span = Math.max(1, Math.round(cell * 0.5));
      // collect r, g, b of every pixel in the window ...
      out.push([median(rs), median(gs), median(bs)]);
    }
  }
  return out;
}
```

2. Classifying the stickers

The classifyScan() function converts the centre sticker of each face into a Lab reference colour. Every sticker is then compared with the six references and receives the letter of the nearest one. A sticker is marked uncertain when it is farther than 45 units from every reference, or when the best and second-best references differ by less than 8 units. The function also warns when two centres are closer than 18 units, because those two colours are likely to be confused.

```
const refs = {};
for (const f of FACES) refs[f] = rgbToLab(captures[f].samples[4]);

const ranked = FACES.map((face) => ({ face, d: deltaE(lab, refs[face]) }))
                    .sort((x, y) => x.d - y.d);
const best = ranked[0], second = ranked[1];
const uncertain = best.d > MAX_CONFIDENT_DISTANCE ||
                  second.d - best.d < MIN_MARGIN;
```

3. Validating the cube

The decodeCube() function checks the 54-sticker state in stages. It first counts the colours and centres. It then maps every corner and edge to a real piece, makes sure no piece appears twice and finds the orientation of each. Finally it checks that the sum of corner twists is divisible by 3, that the sum of edge flips is even, and that the permutation parity of corners and edges is equal. Any failure produces a readable message such as “The Red (Right) colour appears 8 times; it must appear exactly 9 times.”

```
if (cornerOrientation.reduce((a, b) => a + b, 0) % 3 !== 0)
  errors.push('Corner orientation is impossible: at least one corner is twisted.');
if (edgeOrientation.reduce((a, b) => a + b, 0) % 2 !== 0)
  errors.push('Edge orientation is impossible: at least one edge is flipped.');
if (permutationParity(cornerPieces) !== permutationParity(edgePieces))
  errors.push('Corner and edge arrangements have different parity ...');
```

4. Solving and verifying

The solveFacelets() function builds a cubing pattern from the valid state and calls experimentalSolve3x3x3IgnoringCenters(). Before the result is returned, the solution is applied to the pattern and compared with the solved pattern. If it does not end in the solved state, no solution is shown.

```
solution = await experimentalSolve3x3x3IgnoringCenters(pattern);
```

```
const solved = kpuzzle.defaultPattern();
if (!pattern.applyAlg(solution).isIdentical(solved)) {
  return { ok: false, errors: ['The generated solution did not verify ...'] };
}
const moves = [...solution.childAlgNodes()].map((n) => n.toString());
return { ok: true, solution, moves, facelets: encodeCube(pattern.patternData) };
```

5. Hash router

A small custom router reads window.location.hash and re-renders the correct page when it changes, so no routing library is required. A query such as #/scan?face=R opens the Scan page on a chosen face, which is how the Retake button works.

```
export function navigate(route, query) {
  window.location.hash = query
    ? `/${route}?${new URLSearchParams(query)}` : `/${route}`;
}
```

RESULTS

After the interface, vision code and solver were complete and verified in the browser, the application is ready to use. In development it is started with npm run dev and opened at http://127.0.0.1:5173; a production build is created with npm run build and served with npm run preview. The screenshots below were captured from the running application using a fake camera feed and synthetic face photos of a scrambled cube.

After opening the application, it appears like this:

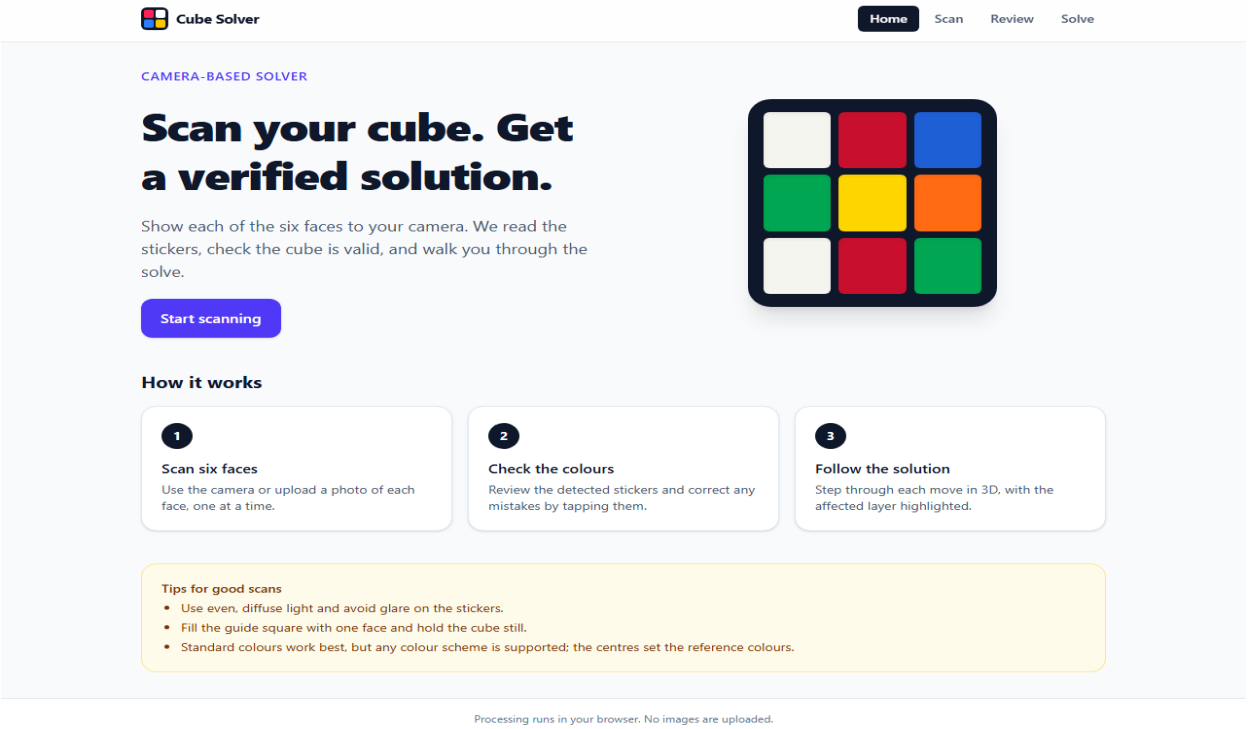


Figure 1: Home page

Scan page with the live camera preview and the 3×3 guide square (a built-in test pattern is shown in place of a real cube):

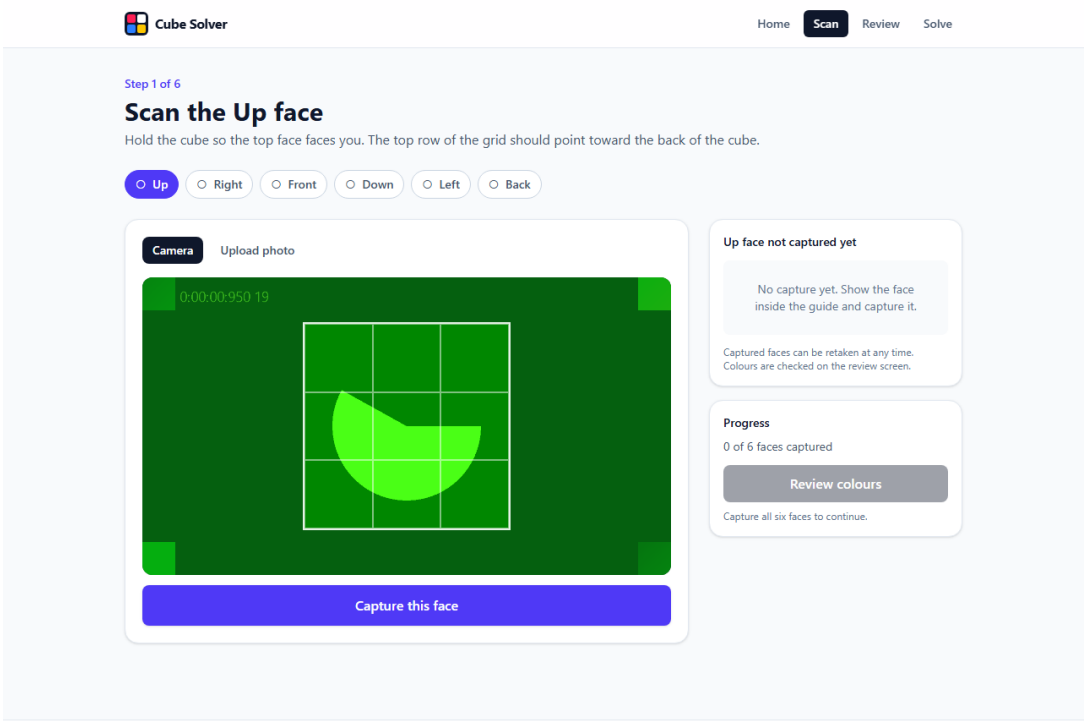


Figure 2: Scan page with live camera preview

Scan page while photos are being uploaded; the captured faces are ticked in the face chips:

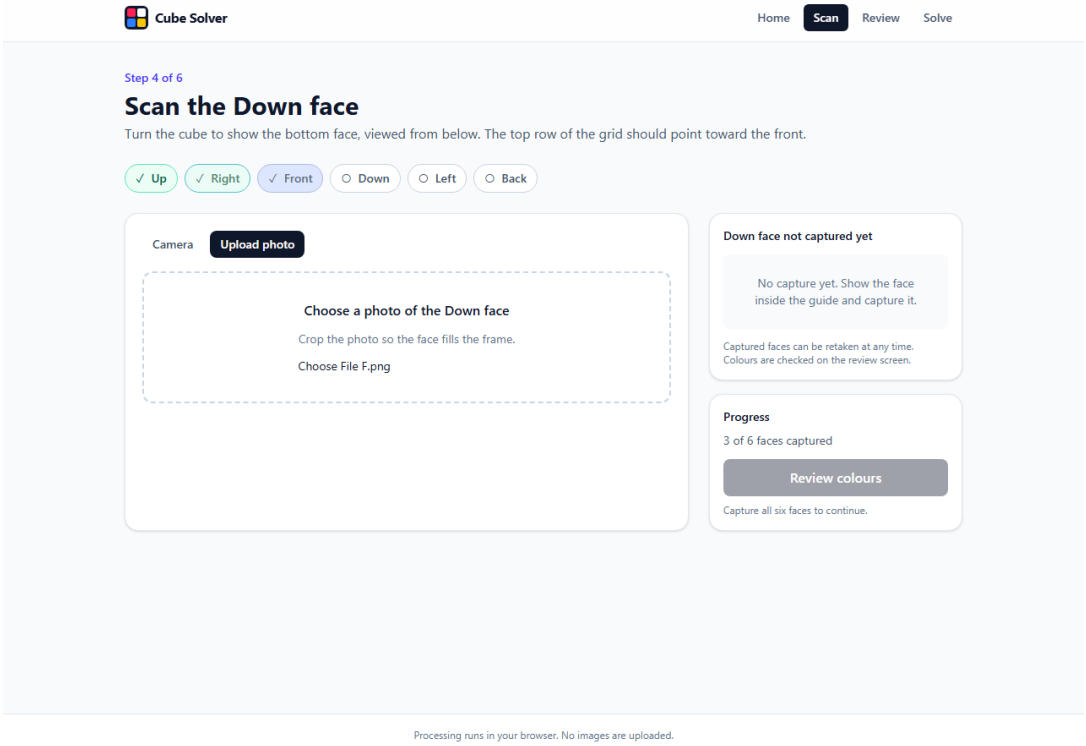


Figure 3: Scanning in progress (three faces captured)

All six faces captured and the Review colours button enabled:

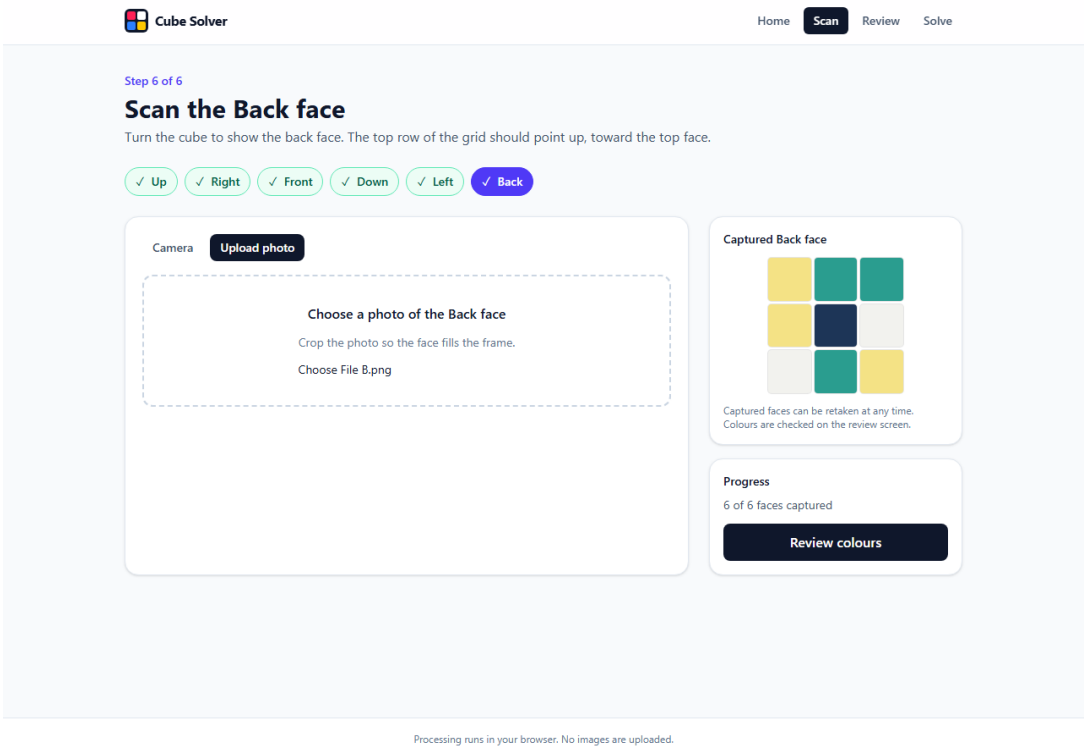


Figure 4: All six faces captured

Review page showing the cube net and a valid cube state:

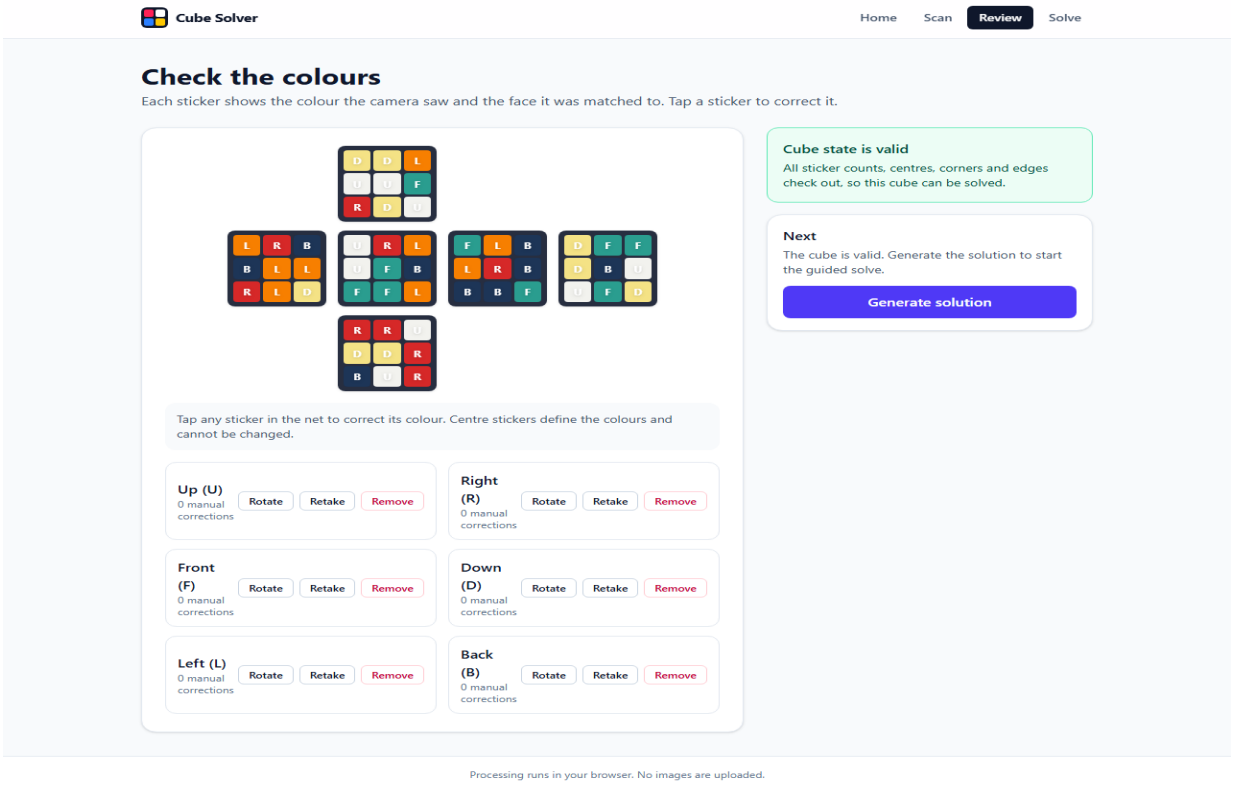


Figure 5: Review page with a valid cube state

Review page when a sticker was read wrongly. The problems are listed and Generate solution is disabled until they are corrected:

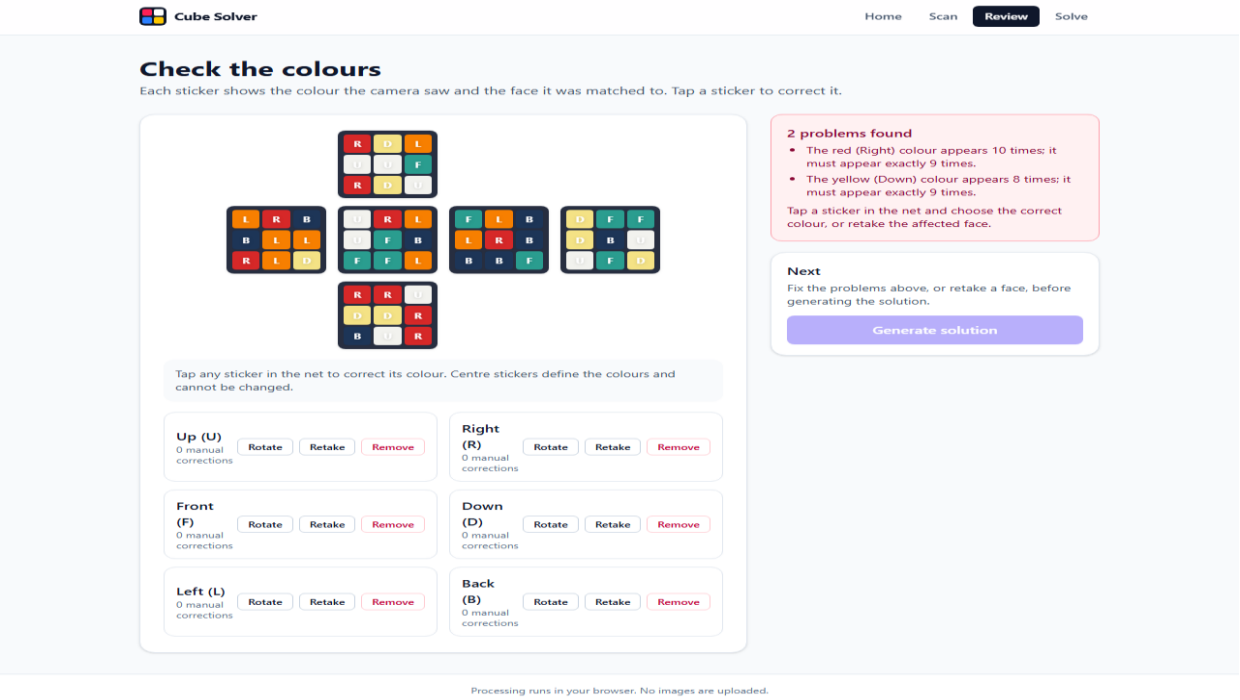


Figure 6: Review page showing validation errors

Solve page at move 0 with the scrambled 3D cube and the generated 18-move solution:

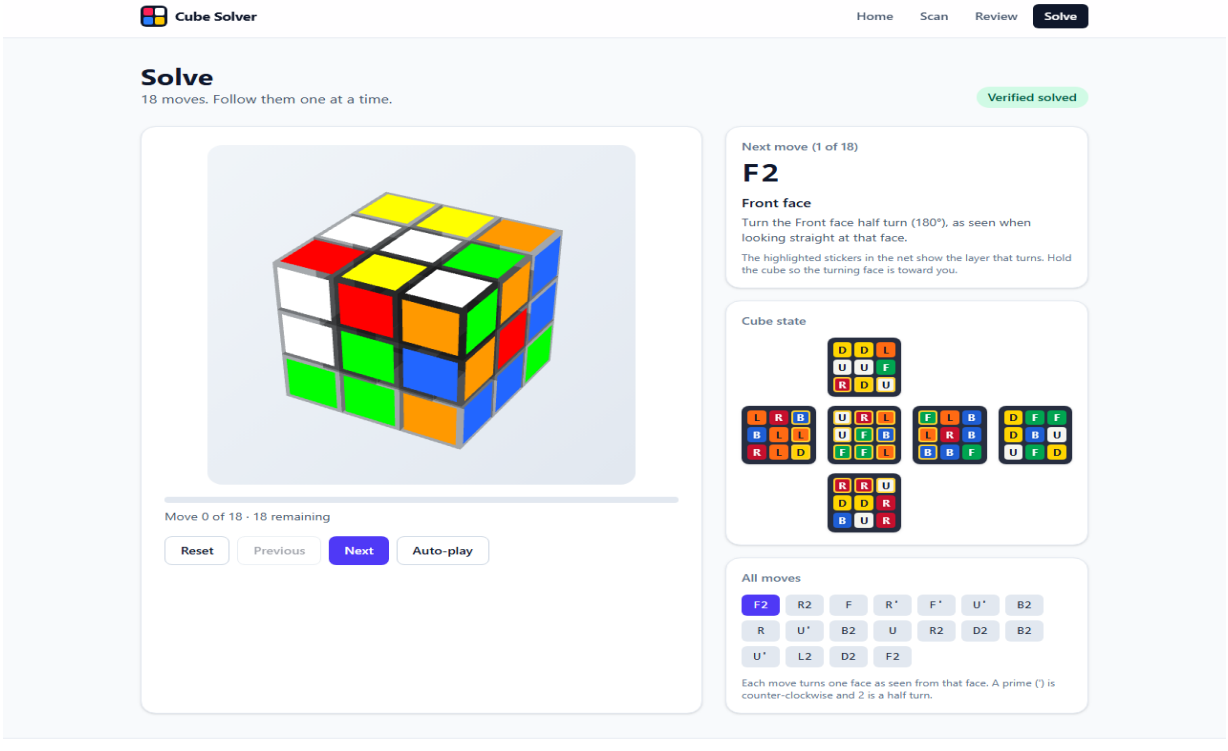


Figure 7: Solve page at the start

Solve page halfway through the solution:

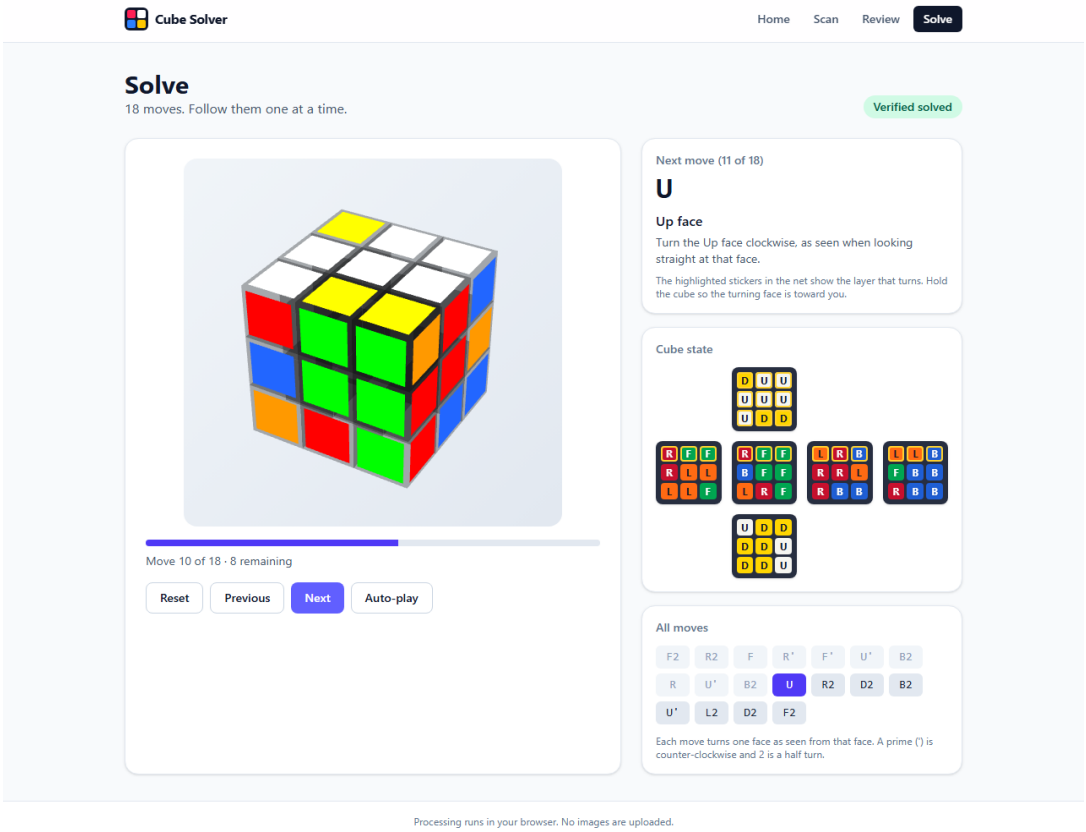


Figure 8: Solve page in the middle of the solution

Solve page after the final move, with the Verified solved badge:

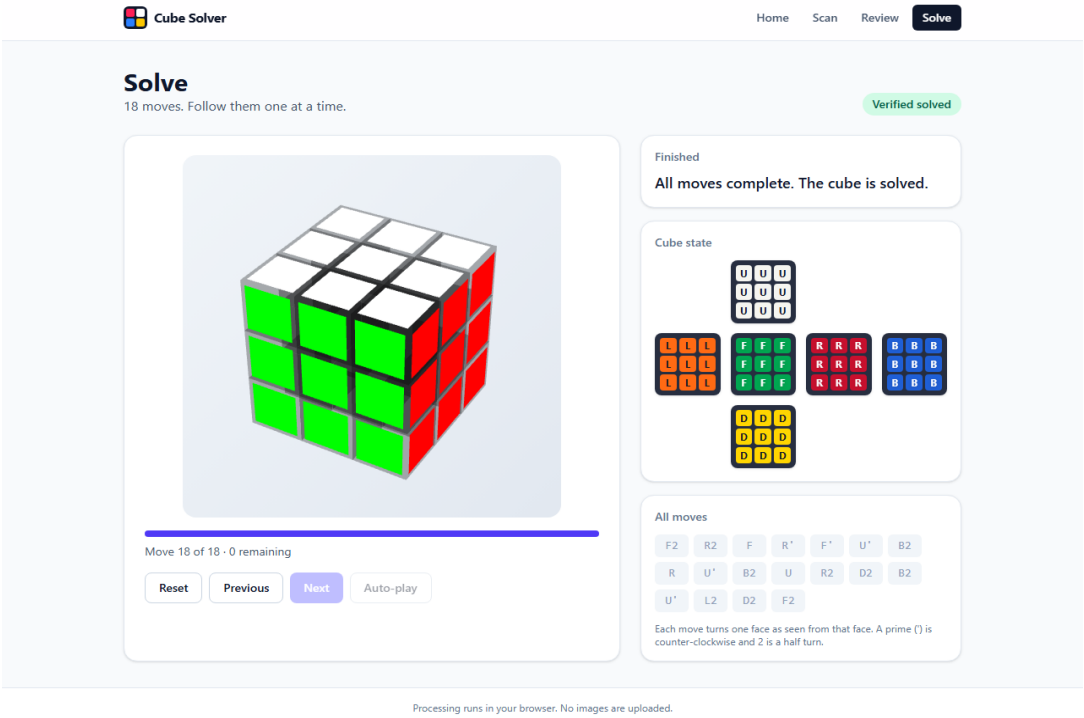


Figure 9: Solve page after the last move (cube solved)

LIMITATIONS

- **Colour recognition is approximate.** It uses sampled colours and CIE76 colour distance, not OpenCV or machine learning. Accuracy depends on lighting, camera white balance and colour similarity. Uncertain stickers are flagged, but accuracy is not guaranteed.
- **Photo upload** assumes the photo is cropped to a single face; the central 90% of the image is sampled.
- **Orientation** depends on holding the cube as the scan step describes. Wrong rotation is usually caught by validation and can be fixed with the Rotate button.
- The solver produces face-turn solutions that may be longer than optimal; computing one typically takes about a second.
- Scan data is kept only for the current browser tab (sessionStorage).
- The 3D view always uses the standard Western colours even when the real cube uses another scheme.
- The application has not yet been tested on real mobile hardware or on iOS Safari.

CONCLUSION

CubeSolve AI shows that a complete scan-to-solution workflow for the Rubik's Cube can run entirely in the browser using modern web technologies. By using the centre stickers as colour references in Lab space, validating the cube mathematically before solving, and verifying every solution, the application stays simple and reliable while remaining private and free. The project also gave the team practical experience with React, Web Workers, camera and canvas APIs, colour science and automated testing.

FUTURE SCOPE

- Automatic detection of the cube position in the camera frame instead of a fixed guide square.
- A more robust classifier (for example CIEDE2000 distance or a small learned model) and automatic glare handling.
- Shorter solutions through an optimal or near-optimal solver, and support for beginner-method explanations.
- Support for other puzzles, such as 2×2×2 or Pyraminx.
- Progressive Web App packaging so that the tool can be installed and used offline on phones.
- Testing and tuning on real mobile devices, including iOS Safari.

REFERENCES

[1] Cubing.js, “cubing” library for twisty puzzles (KPuzzle, TwistyPlayer and 3×3×3 solver), https://js.cubing.net

[2] H. Kociemba, “The Two-Phase Algorithm,” https://kociemba.org/cube.htm

[3] React documentation, https://react.dev

[4] Vite documentation, https://vite.dev

[5] Tailwind CSS documentation, https://tailwindcss.com

[6] CIE 1976 L*a*b* colour space and colour difference (ΔE*ab), International Commission on Illumination.

[7] MDN Web Docs: MediaDevices.getUserMedia(), Canvas API, Web Workers API and Window.sessionStorage, https://developer.mozilla.org

[8] Vitest, https://vitest.dev